

# ALTEN SHOP — État des lieux du projet

Analyse de l'existant : code fourni, technologies, architecture et fonctionnement actuel.

Angular 18

PrimeNG 17

TypeScript 5.4

Signals

Standalone components

Back-end : vide

## 1. Contexte & objectif

Le dépôt est un **test technique de recrutement ALTEN**. La consigne (fichier `README.md` à la racine) décrit un mini site e-commerce, « ALTEN SHOP », pour lequel le candidat doit compléter un front-end Angular et/ou créer un back-end de gestion de produits.

Trois profils sont prévus dans la consigne :

- **Front-end** : enrichir l'application Angular fournie (panier, page contact, etc.).
- **Back-end** : développer une API de gestion de produits (techno au choix) + authentification.
- **Full-stack** : les deux, avec en plus une stratégie de test de l'API.

### À retenir pour l'entretien

Le projet livré est un **squelette de départ** : seul le front est amorcé (CRUD produit partiel), et le back-end n'existe pas encore. Le travail attendu consiste à compléter ce socle.

## 2. Structure du dépôt

```
product-trial-master/
├── README.md           ← cahier des charges (consignes ALTEN)
├── biome.json          ← config lint + formatage (Biome, racine partagée)
├── back/
│   └── .gitkeep        ← DOSSIER VIDE – back-end à créer
├── front/              ← application Angular 18 (le code existant)
│   ├── angular.json    ← config build/serve (configs: mock, production)
│   ├── package.json    ← dépendances + scripts (ng, start)
│   └── src/
│       ├── index.html, main.ts, polyfills.ts, manifest.webmanifest
│       ├── environments/ ← environment.ts (production:false)
│       ├── locale/      ← i18n : translations fr-FR + en-GB (.xlf)
│       ├── styles/      ← thèmes light.css / dark.css, variables.scss, fonts Roboto
│       ├── assets/
│       │   ├── products.json ← 30 produits mockés (données de démo)
│       │   └── icons/       ← icônes PWA (72→512 px)
│       └── app/
│           ├── app.component.* ← layout (toolbar + splitter + menu)
│           ├── app.routes.ts   ← routing racine (home, products, lazy)
│           ├── products/
│           │   ├── data-access/ ← product.model.ts + products.service.ts
│           │   ├── features/product-list/ ← page liste + CRUD
│           │   ├── ui/product-form/ ← formulaire (dialog ajout/édition)
│           │   └── products.routes.ts
│           └── shared/
│               ├── features/home/ ← page d'accueil
│               └── ui/panel-menu/ ← menu latéral
```

### 3. Stack technique (versions exactes)

| Domaine         | Technologie             | Version        | Rôle dans le projet                                                                         |
|-----------------|-------------------------|----------------|---------------------------------------------------------------------------------------------|
| Framework front | Angular                 | 18.0.2         | SPA, composants standalone, routing, DI                                                     |
| Langage         | TypeScript              | 5.4.5          | Typage statique de tout le front                                                            |
| Librairie UI    | PrimeNG                 | 17.18.0        | Composants : Toolbar, Splitter, Card, DataView, Dialog, Dropdown, Inputs, PanelMenu, Button |
| CSS utilitaire  | PrimeFlex               | 3.3.1          | Classes utilitaires (flex, gap, text-center...)                                             |
| Icônes          | PrimeIcons              | 7.0.0          | Pictogrammes (pi pi-shopping-cart...)                                                       |
| Réactivité      | RxJS                    | 7.8.1          | Observables pour les appels HTTP                                                            |
| État (state)    | Angular Signals         | (natif 18)     | Stockage réactif des produits dans le service                                               |
| Graphiques      | Chart.js                | 3.3.2          | Inclus mais non utilisé pour l'instant                                                      |
| Éditeur riche   | Quill                   | 1.3.7          | Inclus mais non utilisé pour l'instant                                                      |
| PWA             | @angular/service-worker | 18.0.2         | Service worker activé (ngsw-config.json)                                                    |
| i18n            | @angular/localize       | 18.0.2         | Traductions fr-FR / en-GB                                                                   |
| Lint / Format   | Biome                   | 1.7.2          | Remplace ESLint+Prettier (config à la racine)                                               |
| Runtime         | Node.js                 | v22.13 (local) | Build et serveur de dev Angular                                                             |

#### Détail qui peut être relevé en entretien

`chart.js` et `quill` sont déclarés dans `package.json` et chargés dans `angular.json` (section `scripts`) mais ne sont **pas encore utilisés** dans le code. Ce sont des dépendances « prêtes à l'emploi » du template de départ.

### 4. Architecture front-end

L'application suit un découpage **par fonctionnalité (feature)** avec une convention de couches qui rappelle les bonnes pratiques Angular modernes :

| Couche             | Dossier                            | Responsabilité                                                                                                                   |
|--------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>data-access</b> | products/data-access               | Modèles ( <code>product.model.ts</code> ) et service de données ( <code>products.service.ts</code> ) — communication HTTP + état |
| <b>features</b>    | products/features, shared/features | Composants « pages » intelligents, routés (product-list, home)                                                                   |
| <b>ui</b>          | products/ui, shared/ui             | Composants présentionnels réutilisables (product-form, panel-menu)                                                               |
| <b>shared</b>      | shared/                            | Éléments transverses (accueil, menu de navigation)                                                                               |

## Layout global

Défini dans `app.component.html` : une **toolbar** en haut (logo + titre « ALTEN SHOP » + icône panier statique), puis un **splitter** 20 % / 80 % : menu latéral à gauche ( `app-panel-menu` ), contenu routé à droite ( `router-outlet` ).

## Routage & lazy loading

```
// app.routes.ts
home      → HomeComponent (changement direct)
products  → loadChildren(...products.routes) // chargé à la demande
""        → redirige vers home

// products.routes.ts
products/list → ProductListComponent
products/**   → redirige vers list
```

Le module produits est **chargé paresseusement** ( `loadChildren` ), bonne pratique pour le découpage du bundle.  
Le menu ( `panel-menu.component.ts` ) expose deux entrées : *Accueil* ( `/home` ) et *Produits* ( `/products/list` ).

## 5. Détail des composants existants

| Composant                         | Type               | Ce qu'il fait                                                                            |
|-----------------------------------|--------------------|------------------------------------------------------------------------------------------|
| <code>AppComponent</code>         | racine, standalone | Layout (toolbar + splitter + menu + outlet). Titre « ALTEN SHOP ».                       |
| <code>HomeComponent</code>        | feature            | Simple carte « Bienvenue sur ALTEN SHOP ! ». Page d'accueil minimale.                    |
| <code>PanelMenuComponent</code>   | ui                 | Menu latéral PrimeNG avec 2 liens (Accueil, Produits).                                   |
| <code>ProductListComponent</code> | feature            | Page liste produits + orchestration du CRUD (création, édition, suppression via dialog). |
| <code>ProductFormComponent</code> | ui                 | Formulaire d'ajout/édition (nom, prix, description, catégorie) dans un dialog.           |

### ProductListComponent — le cœur fonctionnel actuel

- Au `ngOnInit`, appelle `productsService.get()` pour charger la liste.
- Affiche les produits via un `p-dataView` + `p-card` : **seulement la catégorie et le nom** sont affichés (prix, image, stock... non affichés).
- Boutons *Créer, Modifier, Supprimer* ; le formulaire s'ouvre dans un `p-dialog`.
- Gère un signal `editedProduct` et deux drapeaux (`isDialogVisible`, `isCreation`) pour distinguer création et édition.

### ProductFormComponent — formulaire

- Reçoit le produit via `input.required<Product>()` (nouvelle API de signaux d'input).
- Émet `save` et `cancel` via `@Output`.
- Travaille sur une **copie** : `editedProduct = computed(() => ({...product()}))` — évite de muter directement l'objet parent.
- Champs gérés : nom (requis), prix (requis), description, catégorie (dropdown figé : Accessories / Fitness / Clothing / Electronics).

#### Point d'attention technique

Le formulaire ne couvre que 4 champs sur les 14 du modèle `Product` (pas de `code`, `image`, `quantity`, `inventoryStatus`, `rating`, etc.). À compléter selon les besoins du back-end.

## 6. Gestion de l'état — Signals + Service

Le `ProductsService` (fourni `providedIn: 'root'`, donc singleton) centralise l'état avec un **signal privé** exposé en lecture seule :

```
private readonly _products = signal<Product[]>([]);
public readonly products = this._products.asReadOnly();
```

Quatre méthodes HTTP, chacune mettant à jour le signal local après l'appel :

| Méthode               | Appel HTTP                            | Comportement                                                                          |
|-----------------------|---------------------------------------|---------------------------------------------------------------------------------------|
| <code>get()</code>    | GET <code>/api/products</code>        | Si échec → <b>fallback</b> sur <code>assets/products.json</code> . Remplit le signal. |
| <code>create()</code> | POST <code>/api/products</code>       | Si échec → renvoie <code>true</code> . Ajoute le produit en tête de liste.            |
| <code>update()</code> | PATCH <code>/api/products/:id</code>  | Si échec → renvoie <code>true</code> . Remplace le produit dans la liste.             |
| <code>delete()</code> | DELETE <code>/api/products/:id</code> | Si échec → renvoie <code>true</code> . Retire le produit de la liste.                 |

### Piège à connaître (question probable du recruteur)

Le `catchError` de `create/update/delete` **masque les erreurs réseau** et renvoie toujours « succès », puis met quand même à jour l'état local (mise à jour « optimiste »). C'est pratique pour démonter le front *sans* back-end (tout marche en mémoire), mais en production cela ferait croire à l'utilisateur qu'un enregistrement a réussi alors qu'il a échoué. À corriger une fois le vrai back branché.

## 7. Modèle de données

```
interface Product {
  id: number;
  code: string;
  name: string;
  description: string;
  image: string;
  category: string;
  price: number;
  quantity: number;
  internalReference: string;
  shellId: number;
  inventoryStatus:
    "INSTOCK"|"LOWSTOCK"|"OUTOFSTOCK";
  rating: number;
  createdAt: number;
  updatedAt: number;
}
```

Le modèle `Product` (côté front) est **identique** à celui imposé par la consigne back-end du README. C'est le contrat partagé entre front et futur back.

Les données de démo (`assets/products.json`) contiennent **30 produits** (montres, vêtements, accessoires...) avec ces champs renseignés, ids à partir de `1000`.

Note : `quantity` existe dans le modèle mais n'est pas affiché ni édité actuellement — utile pour le futur panier et la gestion de stock.

## 8. Outillage & configuration

| Élément           | Détail                                                                                                                                                                                                            |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scripts npm       | <code>npm start</code> = <code>ng serve</code> (serveur de dev). Pas de script de build/test explicite ajouté.                                                                                                    |
| Configs de build  | <code>mock</code> (remplace l'environnement par <code>environment.mock.ts</code> ) et <code>production</code> (optimisation, hashing, budgets 2 Mo / 6 Mo).                                                       |
| Lint / Format     | Biome ( <code>biome.json</code> ) — alternative tout-en-un à ESLint + Prettier, indentation espaces, point-virgules obligatoires.                                                                                 |
| PWA               | Service worker activé ( <code>serviceWorker:true</code> , <code>ngsw-config.json</code> ) + manifest + icônes 72→512 px. App installable.                                                                         |
| i18n              | Locale fr enregistrée dans <code>main.ts</code> ; fichiers de traduction <code>fr-FR</code> et <code>en-GB</code> .                                                                                               |
| Thèmes            | Deux thèmes PrimeNG ( <code>light</code> / <code>dark</code> deeppurple) chargeables dynamiquement ( <code>inject:false</code> ).                                                                                 |
| Providers globaux | <code>main.ts</code> : HttpClient, animations, router, et services PrimeNG ( <code>MessageService</code> , <code>ConfirmationService</code> , <code>DialogService</code> ) — prêts pour les toasts/confirmations. |

### Opportunité visible

`MessageService` et `ConfirmationService` sont déjà fournis globalement : on pourra afficher des *toasts* (« Demande de contact envoyée avec succès », confirmation de suppression...) sans configuration supplémentaire.

## 9. Le back-end

### État : inexistant

Le dossier `back/` ne contient qu'un fichier `.gitkeep` (placeholder Git). **Aucune ligne de code serveur.** Tout est à créer : techno, API, base de données, authentification.

Conséquence : aujourd'hui, l'application fonctionne **uniquement en mode mock**. Les appels `/api/...` échouent et le service bascule sur le JSON local. La persistance n'est donc pas réelle (rechargement de page = retour aux 30 produits d'origine).

## 10. Récapitulatif : ce qui marche / ce qui manque

### DÉJÀ FONCTIONNEL

- ✓ Démarrage de l'app, layout, navigation (Accueil / Produits)
- ✓ Chargement des produits (fallback JSON)
- ✓ Affichage liste (nom + catégorie)
- ✓ Créer / Modifier / Supprimer un produit (en mémoire)
- ✓ Formulaire dans un dialog

### MANQUANT (À DÉVELOPPER)

- ✗ Panier (service, ajout, suppression, badge, page)
- ✗ Affichage produit complet (prix, image, stock...)
- ✗ Page & menu « Contact » + formulaire validé
- ✗ Pagination / filtrage (bonus)
- ✗ Ajustement des quantités (bonus)
- ✗ **Tout le back-end** (API, JWT, admin, cart, wishlist)
- ✗ Tests (aucun `.spec` présent)

- ✓ PWA, i18n, thèmes, lint configurés

## 11. Points d'attention à savoir défendre en entretien

| Sujet                  | Ce qu'il faut pouvoir dire                                                                                                                                                                            |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pourquoi des Signals ? | État réactif natif Angular 16+, plus simple que NgRx pour ce périmètre, granularité fine, pas de boilerplate. Le service expose un état en lecture seule ( <code>asReadonly</code> ) → encapsulation. |
| Standalone components  | Plus de NgModules : chaque composant déclare ses propres imports. Recommandé depuis Angular 17, simplifie l'arbre de dépendances et le lazy-loading.                                                  |
| Le fallback JSON       | Permet de développer le front sans attendre le back. Mais le <code>catchError</code> sur les mutations masque les vraies erreurs → à durcir en prod (remonter l'erreur, toast d'échec).               |
| Lazy loading           | <code>loadChildren</code> sur la feature produits → le code n'est téléchargé qu'à la navigation. Bon pour le temps de chargement initial.                                                             |
| PrimeNG vs maison      | Gain de temps (composants riches accessibles prêts à l'emploi : DataView, Dialog, Dropdown). Cohérent avec un contexte ESN/livraison rapide.                                                          |
| Biome                  | Outil unique (lint+format) écrit en Rust, très rapide ; remplace ESLint+Prettier. Montre une stack outillage moderne.                                                                                 |

Document 1/2 — État des lieux. Voir le document 2 « Attendus & choix techniques » pour le plan de réalisation et les justifications à présenter au recruteur.